



Katholieke
Universiteit
Leuven

BIKING INFRASTRUCTURE MAINTENANCE PROJECTION

A Modern Data Analytics Project

Group 13

Wing Fung Wingo Ng (r1082613)

Rin Yoshida (r1069065)

Dhruv Gupta (r1077995)

Manasvi Kamal Ghelani (r1077547)

25 May 2026

1 Introduction

Biking culture is pertinent to life in Flanders, making the maintenance of cycling infrastructure absolutely critical. For AWW (Agentschap Wegen en Verkeer), the overseeing road management body, identifying which routes require urgent intervention is a constant challenge especially when factoring in Flanders' volatile shower-prone weather. These maintenance operations must not disrupt the public and roadworks should ideally be restricted to low-traffic periods.

This project delivers an end-to-end forecasting solution that predicts which sites urgently require maintenance and identifies the optimal windows for execution of the same over the next seven days. We trained this forecaster and built a centralized monitoring dashboard designed for AWW ML engineers to track pipeline health, data integrity, and predictive performance at a glance.

2 Data Pipeline Architecture

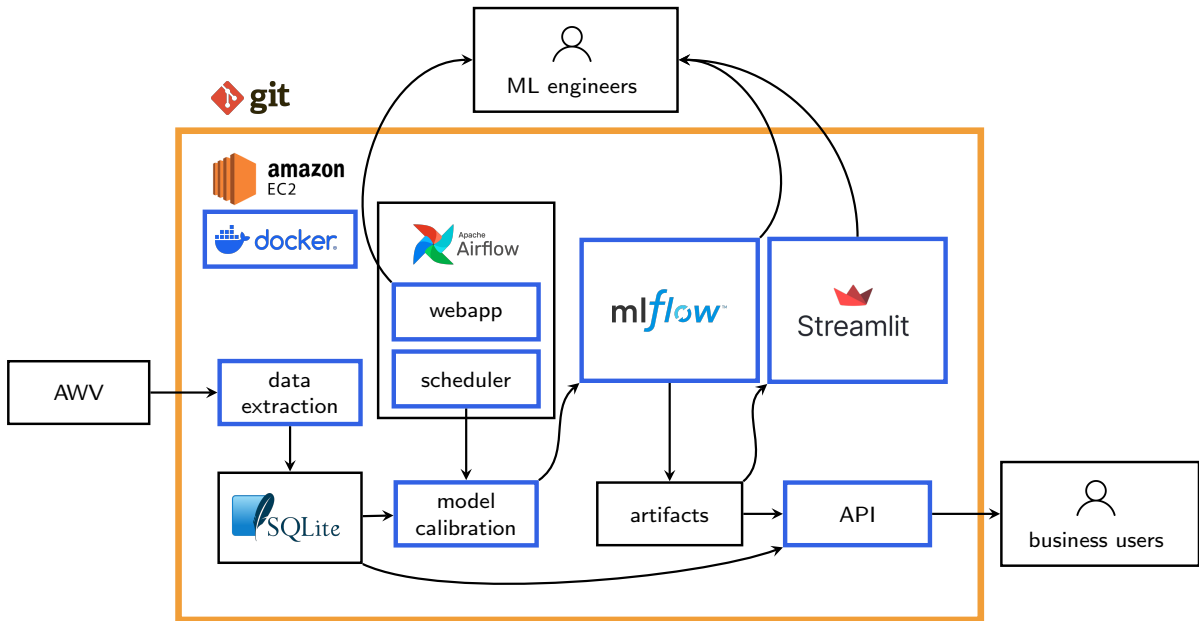


Figure 1: Data pipeline architecture

Figure 1 shows our design of the data pipeline architecture. It can be divided into four parts: (i) data extraction from AWW to SQLite; (ii) model calibration scheduled with Airflow and monitored by MLflow to store resulting artifacts; (iii) three dashboards including Airflow, MLflow and Streamlit to be monitored by ML engineers; and (iv) an API to be consumed by business users. Each service is containerized in a specific docker container to be hosted on AWS EC2 virtual machine. The entire codebase is maintained with version control using git. In the following subsections, we discuss and motivate our choice of each tech stack component in more details.

2.1 SQLite: Data Extraction

To build a reliable and automated platform, the data collection process has been transitioned from manual downloads into a simple, programmatic python script. This script interacts directly with the AWV data portal to harvest the necessary dataset for the model to run and track.

2.1.1 Data Schema and Target Variables

The extraction engine targets monthly CSV files hosted on the AWV server and formatted as data-YYYY-MM.csv. Since these files lack an embedded header row, the script automatically applies a standardized relational schema during ingestion, converting the raw data into structured tabular data. Table 1 shows the data schema of the table in the database.

Table 1: Data schema of the table in the database

Column Name	Data Type	Description
Site_ID	TEXT	Unique identifier for the physical loop sensor
Direction	TEXT	Direction of bicycle travel (e.g., IN, OUT)
Modus	TEXT	Mode of transport (strictly <i>FIETSEERS</i>)
Start_Time	TEXT	Start of the 15-minute sampling window
End_Time	TEXT	End of the 15-minute sampling window
Count	INTEGER	Total volume of bicycle crossings

2.1.2 Incremental Loading Strategy

A simple data extraction script would download all 7 plus years of historical data every single time it runs, wasting server bandwidth and computational time. To solve this, the pipeline implements an incremental loading strategy:

When executed, the script opens the local SQLite database, say *fietstellingen.db* and runs a query: `SELECT MAX(Start_Time)` to find the single most recent timestamp already saved. If no database exists, it defaults to a baseline of 2019-08-01. To handle daily pipeline runs mid-month, the script generates a comprehensive daily sequence spanning from that latest database timestamp up to the current calendar date. Because the raw data files are partitioned by month on the remote server, the script extracts only the unique YYYY-MM month strings from that daily sequence. This ensures that the engine only requests the specific monthly files containing non-ingested days, hitting the network exactly once per file.

2.1.3 Duplicate Check via INSERT or IGNORE

Since the AWW server gets updated on a daily basis, re-downloading the active month’s CSV to capture yesterday’s new records introduces a major data engineering challenge, which is overlapping. Identical timestamps will trigger massive duplication or violate standard database constraints. To guarantee that the database state remains identical whether the pipeline runs once or one hundred times, a two-layer database firewall is integrated directly into the data extraction script. First of all, before data ingestion begins, an initialization function programmatically establishes a composite unique index across the logical keys of the schema created. Then, the pipeline overrides the standard Pandas engine with a custom insertion handler utilizing SQLite’s *INSERT OR IGNORE* syntax.

2.2 LightGBM: Machine Learning Model for Forecasting

2.2.1 Data Preparation

To train the prediction model, bicycle count data were extracted from the SQLite database using SQL queries. Specifically, data covering one year of training data, one month of test data, and an additional seven days required for lag feature construction were retrieved. Within the SQL query, the total number of bicycles counted was aggregated at the daily level for each site. If all records for a given site on a given day were missing, the aggregated daily count was returned as null rather than zero, so that missing observations could be distinguished from true zero counts. As the goal of this project is to predict overall bicycle traffic volume at each site, the direction of bicycle travel was not considered, and counts from all directions were combined.

Additionally, since the database continuously receives new records up to the current day, the most recent data may be incomplete due to delayed ingestion. Some of the latest records were found to contain incomplete or abnormally low counts. To reduce the risk of using unstable data, the extraction process excluded the most recent two days.

Moreover, missing daily counts were subsequently replaced with the mean bicycle count from the previous seven days for the corresponding site. This approach was used to preserve temporal continuity while minimizing information loss caused by incomplete observations.

Besides the location information and the date, some additional explanatory variables were created in order to capture temporal patterns, seasonality, public holidays, and recent historical traffic behavior. Table 2 is the summary of the features.

Table 2: Description of Variables Created

Variable	Type	Description
dayofweek	int	Day of the week (0–6)
is_weekend	binary	Whether the day falls on a weekend
Is_Public_Holiday	binary	Whether the day is a Belgian public holiday
month_sin	float	Sine-transformed month value
month_cos	float	Cosine-transformed month value
lag_1	int	Total bicycle count at the same site, 1 day earlier
lag_2	int	Total bicycle count at the same site, 2 days earlier
lag_3	int	Total bicycle count at the same site, 3 days earlier
lag_7	int	Total bicycle count at the same site, 7 days earlier

2.2.2 Model Development

LightGBM was selected as the prediction model for this project. Since bicycle traffic has temporal patterns and seasonality, time series models such as SARIMA were initially considered. However, it would require fitting separate models for each site, making the approach less scalable for a multi-location deployment setting. In addition, preliminary model comparison showed that LightGBM achieved better predictive performance. LightGBM was therefore adopted as the final model.

To allow the model to capture temporal dependencies similarly to auto-regressive models, lag-based features were explicitly engineered and included as predictors. The `site.id` variable was treated as a categorical feature so that the model could learn site-specific traffic patterns.

The training set consisted of one year of historical data, while the test set covered one month immediately following the training period. The split was performed chronologically rather than randomly in order to avoid data leakage.

Since the goal of the project is to forecast bicycle traffic for future dates, only information that would realistically be available at prediction time can be used. For future periods, actual bicycle counts are unknown, meaning only date-related information and previous observations are available. To address this, a recursive forecasting approach was implemented. Starting from the day immediately after the final training observation, date-related features were generated for the new prediction date, and the model produced a predicted bicycle count. This predicted value was then appended to the historical dataset and used as an input for generating lag features for the following day. This process was repeated iteratively until forecasts for the full one-month horizon were obtained.

A known limitation of this recursive approach is that prediction errors may accumulate over time, since forecasted values are reused as inputs for subsequent predictions.

2.2.3 Model Evaluation

Model performance was assessed using Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE), which were calculated by comparing predicted bicycle counts against observed values in the test dataset.

2.3 MLflow: Machine Learning Monitoring

To ensure accountability, and seamless operations, the training pipeline integrates an enterprise MLOps framework powered by *MLflow*. Instead of treating model training as a simple and naive script execution, every calibration run is monitored, logged, and evaluated against strict quality checks before promotion to production.

The MLflow tracking server sits central to our model management workflow. It operates inside a containerized network endpoint (*http://mlflow:5000*) backed by a persistent relational meta-data store (*sqlite:///mlflow.db*). It captures training conditions including historical context windows, cutoff dates, data parameters, and other features. Additionally, it records quantitative model errors generated during the recursive forecasting execution to visually evaluate every calibration run.

2.3.1 Experiment Lifecycle & Logged Parameters

The pipeline registers run under a centralized experiment cluster: Forecaster Calibration. Figure 2 shows a screenshot of the overview of a specific run.

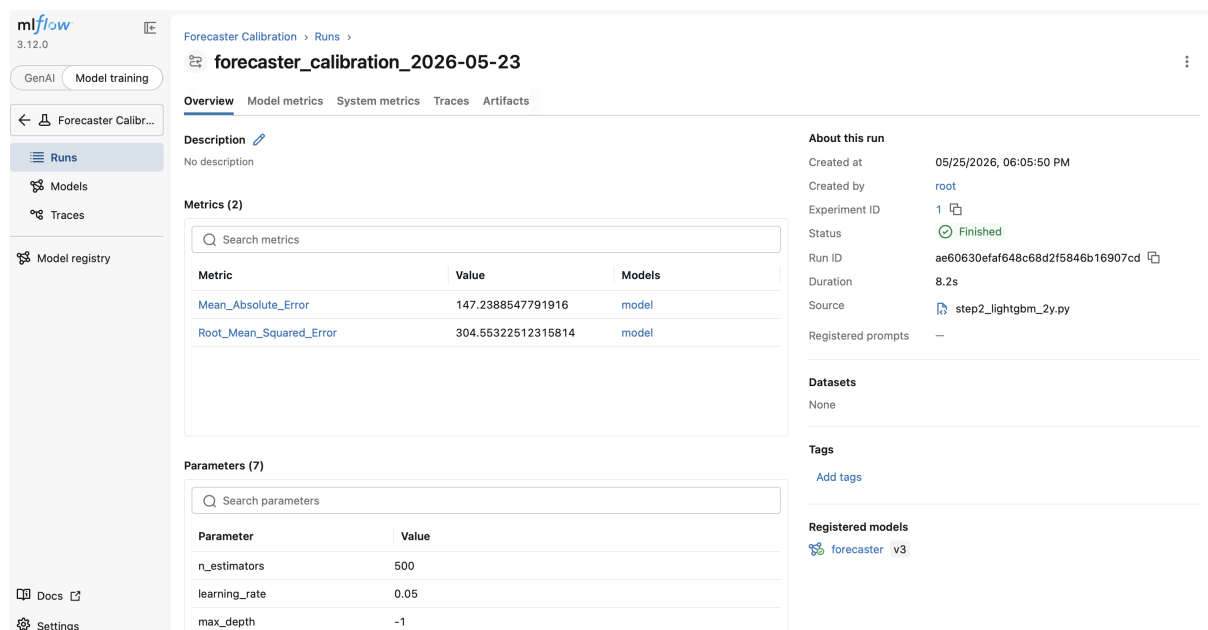


Figure 2: Overview of a forecaster calibration run

Every specific run execution automatically isolates and logs three structural layers of information:

1. **Hyperparameters and pipeline configuration:** The system logs the structural configuration parameters utilized by the LightGBM estimator alongside historical timeline definitions to contextualize model performance:
 - `n_estimators`: 500 (The total number of sequential decision trees compiled)
 - `learning_rate`: 0.05 (The step-size shrinkage factor used to prevent overfitting)
 - `num_leaves`: 31 (The maximum tree leaves allowed per base learner)
 - `train_days`: The exact count of historical training days used (e.g., 365 or 730)
 - `cutoff_date`: The definitive temporal boundary splitting training history from the validation window.
2. **Recursive forecast evaluation metrics:** Rather than calculating standard static evaluation metrics, errors are derived dynamically through a **`recursive_forecast_lgbm()`** function where model predictions feed back as autoregressive lags (`lag_1` all the way to `lag_7`). The aggregate deviations are tracked over a 30-day testing window using two key metrics:
 - **Mean absolute error (MAE):** Represents the expected average daily prediction variance per site in absolute bicycle counts.
 - **Root mean squared error (RMSE):** Accentuates and penalizes large residual tracking mistakes caused by extreme weather patterns or holiday anomalies.
3. **Quality Checks and CD Model Promotion:** To automate Continuous Delivery (CD) without manual inspection, the framework incorporates an automated model registry with programmatic promotion checks. This ensures that underperforming models are safely isolated, while high-performing models are instantly deployed. If the model satisfies the MAE and RMSE threshold boundaries, the serialized binary file is pushed directly to the central directory under the identifier name `forecaster`. Using the `MlflowClient()` API, the script updates a production-ready pointer known as an alias, which is the champion tag to the newly registered version, in this case.

2.4 Git: Version Control

To ensure smooth collaboration across a multi-person engineering team, track changes transparently, and establish a foundational baseline CI/CD, the entire project infrastructure is managed via a centralized, secure version control system utilizing Git and hosted on GitHub within the repository *mda-cycling*, <https://github.com/winglab/mda-cycling>.

The repository's structural organization separates operational code, analytical experiments, pipeline DAGs, and infrastructure configurations into isolated domains. This

clean separation of concerns prevents conflicts during concurrent developer updates. Table 3 shows the files and folders organized in the repository *mda-cycling*, together with the purpose of each file or folder.

Table 3: Repository of *mda-cycling*

File/Folder	Remarks
.gitignore	Excludes temp files, local DBs, and mlruns from tracking
Dockerfile	Blueprint for the core Python & LightGBM system execution image
README.md	System documentation, onboarding guide, and setup instructions
docker-compose.yml	Production multi-service container mesh configuration
requirements.txt	Core Python library dependencies and pin constraints
dags/	Directed Acyclic Graphs (DAGs) for Apache Airflow scheduling
src/	Core production source scripts (extraction, training, business APIs)
notebooks/	Local exploratory data analysis and initial ML prototyping

Collaborative data engineering projects face unique challenges, such as large data binaries or local run folder generation tracking over and mutating shared project histories. For an instance, large database structures, such as *fietstellingen.db*, temp execution files (*.csv exports*), and local tracking storage folders (*mlruns/*, *logs/*, *.pytest_cache*) are explicitly blacklisted from tracking using a strictly defined *.gitignore* file. This guarantees that developers only commit clean, functional code logic without polluting the remote repository with heavy data artifacts. Additionally, commits are logged with explicit context. This semantic tracking timeline enables the team to trace performance changes or pipeline modifications directly back to specific code commits, simplifying debugging and code rollbacks if a newly deployed feature degrades model performance.

Maintaining this explicit repository architecture directly simplifies the downstream deployment pipeline on the production server. Because all environmental requirements (*requirements.txt*), system instructions (*Dockerfile*), and multi-container connection definitions (*docker-compose.yml*) are versioned as code. Any team member or external deployment server can clone the repository, run a single docker compose up –build command, and replicate this identical automated setup instantly. Also, this code structure serves as the foundation for modern git operations. When code is pushed to the main branch, it can automatically trigger webhooks or CI/CD runners such as GitHub Actions, to run unit tests on the extraction script, rebuild the container images, and dynamically pull the fresh configurations directly onto the target server without causing operational downtime.

2.5 Airflow: Workflow Scheduling

As a tool to orchestrate the daily forecasting pipeline while managing the dependencies between tasks, Airflow is used. In this workflow, the data extraction task is executed first, followed by the prediction task, ensuring that the forecasting model always uses the most recently available data. This also makes the execution order and task dependencies

explicit. Figure 3 shows the directed acyclic graph (DAG) on Airflow dashboard that describes the tasks of the daily forecasting pipeline.

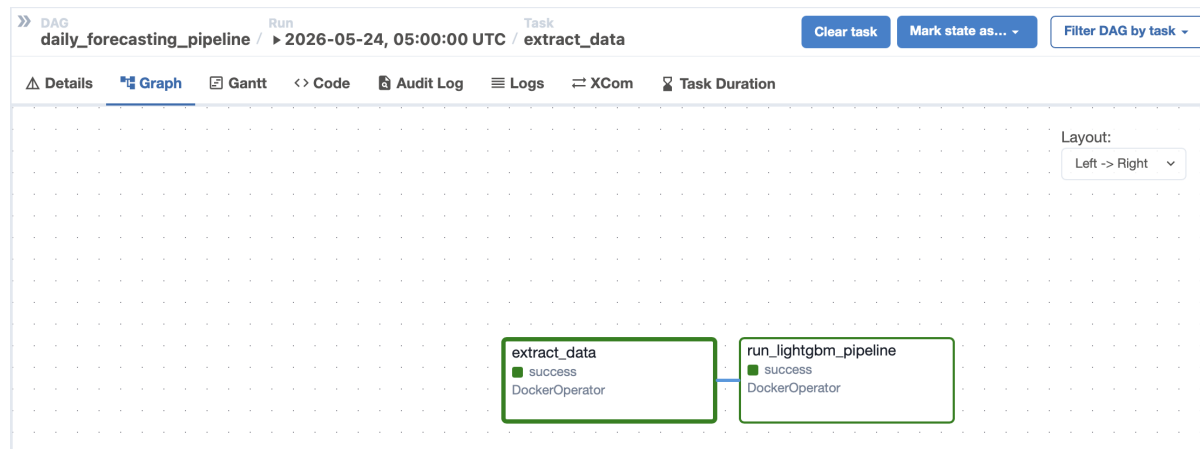


Figure 3: Directed acyclic graph demonstrating the workflow of tasks

The DAG is scheduled to run every morning at 05:00 UTC, which corresponds to 07:00 in Belgium during summer time. This allows the extraction and forecasting process to be completed before the system users, such as ML engineers or analysts, start their work.

If a task fails, Airflow retries it once after a delay of five minutes. The number of retries is limited to one to avoid repeatedly running a failing pipeline. Airflow also provides task-level logs through its dashboard, making it possible to identify which step failed and inspect the corresponding error message. The logs can be found under the tab *Logs* for each task. Figure 4 shows a screenshot of the logs generated for the task *extract_data*.

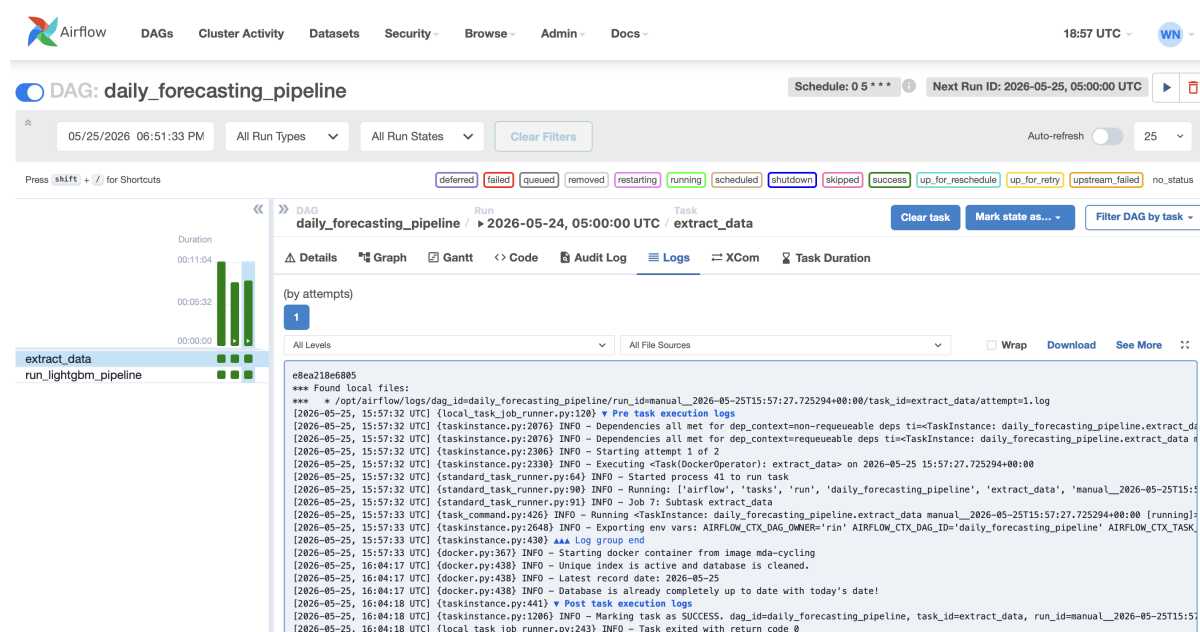


Figure 4: Logs generated for the task *extract_data*

2.6 Streamlit: Interactive Monitoring Dashboard

Our comprehensive [Streamlit dashboard](#) facilitates real-time health monitoring for both the data pipeline and the underlying LightGBM model. If an anomaly arises, the dashboard immediately flags the issue, allowing the ML engineer to isolate and investigate it. The test cases for this monitoring purpose are listed in Table 4 below:

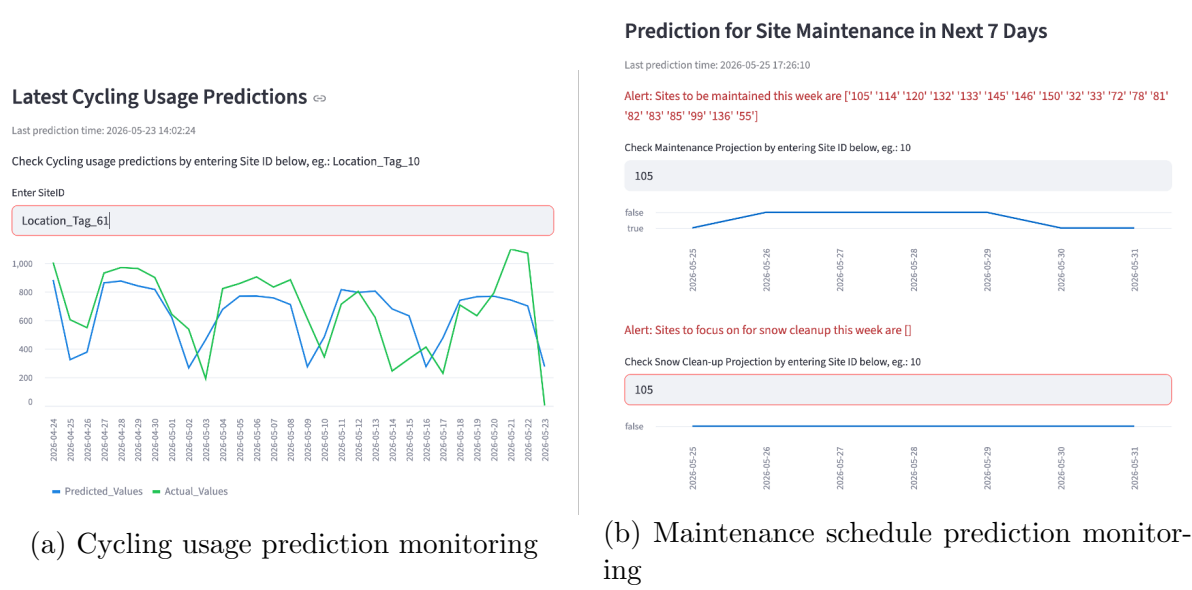
Table 4: Test Cases monitored on the dashboard

Test Case	Description
Cycling traffic prediction time generation	Verifies the timestamp of the latest automated traffic forecast generation.
Data completeness check for cycling traffic prediction	Validates that traffic predictions exist across all active monitoring sites and target dates.
Model error tracking	Displays real-time Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) metrics across model runs.
Site-specific cycling usage	Visualizes predicted vs. actual cycling utilization for granular model validation.
Maintenance prediction time generation	Verifies the timestamp of the latest maintenance scheduling execution.
Data completeness check for maintenance prediction	Validates that a finalized maintenance timeline exists across all active sites and target dates.
Site-specific maintenance schedule	Renders interactive operational maintenance windows for field-team planning.

The pipeline automatically queries a [weather API](#) to fetch 7-day snowfall forecasts. If snowfall is predicted for any site, the dashboard displays an alert for road snow removal. For standard, non-weather-related upkeep, the dashboard calculates ideal low-traffic windows by combining the LightGBM model’s cycling usage predictions with the weather data. The maintenance scheduling logic used at present is:

1. **Routine Maintenance:** The suggested windows are restricted to days where predicted traffic falls below 10% of the typical daily traffic baseline for that site.
2. **High-Priority Snow Clean-up:** On days with active snowfall predictions, the site is automatically escalated to high priority. The same low-traffic threshold logic is applied to find the safest, least disruptive window to deploy snowplows.

Figure 5: Overview of the Streamlit dashboard components



The dashboard automatically fetches fresh model predictions and refreshes every 30 minutes. If metrics are not consistent or a pipeline step fails, direct links to integrated MLflow and Apache Airflow dashboards are provided on the sidebar to facilitate the engineer to seamlessly dive into debugging.

2.7 Docker: Service Containerization

There are a total of seven services, each containerized in their own docker. The seven services include (i) data extraction; (ii) model calibration; (iii) Airflow dashboard; (iv) Airflow scheduler; (v) MLflow; (vi) Streamlit dashboard; and (vii) API for business users. The advantage of having each service in their own container is that they can be managed separately. It also ensures that the each service would not interfere with one another. For example, if the Streamlit dashboard is down, all other services like Airflow dashboard, API to business users, etc. are still running as normal in their own container. Figure 6 shows a screenshot of a list of running containers and their process status. Another advantage of containerization is the portability. The containers are portable across different platforms and thus making it easier for deployment and migration.

```
[ec2-user@ip-172-31-25-133 ~]$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
dd7bf3a3d9a4   apache/airflow:2.9.3                "/usr/bin/dumb-init ..." 21 hours ago   Up 21 hours   0.0.0.0:8080->8080/tcp, :::8080->8080/tcp   mda-cycling-airflow-server-1
ed036dd4c2f5   apache/airflow:2.9.3                "/usr/bin/dumb-init ..." 21 hours ago   Up 21 hours   8080/tcp                               mda-cycling-airflow-scheduler-1
73ebd06f1ed4   mda-cycling                          "python -m uvicorn s..." 21 hours ago   Up 21 hours   0.0.0.0:8000->8000/tcp, :::8000->8000/tcp   mda-cycling-business-api-1
cbd713ef5b76   ghcr.io/mlflow/mlflow               "mlflow server --hos..." 21 hours ago   Up 21 hours   0.0.0.0:5000->5000/tcp, :::5000->5000/tcp   mlflow
```

Figure 6: docker ps - a list of containers with their process status

There are many services that need to be set up on the production machine and it is time consuming and error prone to set them up one by one. This leads us to the use of *docker-compose.yml* file. All the services and configurations are included in a single

file. One can then use the command *docker compose up -d* to start all the services in the background. To bring down all the services, one can use *docker compose down*. This provides us with an easy way to manage a collection of different services.

Each container is separate, but sometimes they need to reference to the same data store. For example, the data extracted by the data extraction service will be used in the model calibration service for training the model. This is how *volume* comes into the picture. Volumes are persistent data stores for containers. Figure 7 shows a list of volumes. In this case, the data extracted by the data extraction service would be stored in the volume *mda-cycling_mda* and later read by the model calibration service when training the model.

```
[[ec2-user@ip-172-31-25-133 ~]$ docker volume ls
DRIVER      VOLUME NAME
local       mda-cycling_airflow_home
local       mda-cycling_mda
```

Figure 7: docker volume ls - a list of volumes

On the other hand, at times there is a need for different containers to communicate with each other. For example, the API for business users would need to go through the MLflow server to load the latest production model for forecasting. This motivates us to establish a *network* in docker. In this case, all the services are set up in the same network called *mda-net*, so that they could communicate with one another through the exposed ports, while operating independently.

2.8 API: End Point for Business Users

We provide an API for our downstream business end users to consume our forecasts. They could use the web browser and see the results through a link. For example, if one would like to get the forecasts for location tag 1 from 2026-05-24 to 2026-06-23, one can use `http://<server-ip>/getForecasts?loc=1&start=2026-05-24&end=2026-06-23` to get the forecasts. Figure 8 shows a sample response of the same data request. Business users can then proceed to make maintenance scheduling decisions based on our forecasts. For example, they could choose the dates with least expected traffic to minimize the impact to cyclists for regular maintenance work.

It is also worth noting that such kind of data requests can easily be done in Python. Thus, if they would like to for example build an interactive dashboard to visualize the data, this could easily be achieved on their end as well.

```
{"loc": "Location_Tag_1", "start": "2026-05-24", "end": "2026-06-23", "forecasts": {"2026-05-24": 129, "2026-05-25": 272, "2026-05-26": 386, "2026-05-27": 426, "2026-05-28": 438, "2026-05-29": 465, "2026-05-30": 313, "2026-05-31": 359, "2026-06-01": 525, "2026-06-02": 594, "2026-06-03": 566, "2026-06-04": 535, "2026-06-05": 511, "2026-06-06": 321, "2026-06-07": 354, "2026-06-08": 545, "2026-06-09": 629, "2026-06-10": 604, "2026-06-11": 567, "2026-06-12": 535, "2026-06-13": 314, "2026-06-14": 360, "2026-06-15": 566, "2026-06-16": 626, "2026-06-17": 596, "2026-06-18": 545, "2026-06-19": 536, "2026-06-20": 320, "2026-06-21": 354, "2026-06-22": 568, "2026-06-23": 628}}
```

Figure 8: A sample of HTTP GET request response for the API getForecasts

2.9 AWS EC2: Deployment to Virtual Machine on Cloud

All the services are hosted on a virtual machine on AWS EC2 for production. AWS EC2 is an IaaS that allows us to rent secure virtual machines on the cloud without the need to manage the hardware and infrastructure on our own.

Some of the services in fact could be set up using other Amazon Web Services. For example, MLflow could be set up using SageMaker, Airflow could be set up using Amazon Managed Workflows for Apache Airflow (MWAA) and API for business users could be set up using AWS Lambda. However, for the simplicity of this project, we choose to deploy all services onto a single EC2 virtual machine. Note that if the scope of project is going to be expanded, it would be advantageous to migrate to the aforementioned Amazon Web Services for scalability reasons.

As all the required set-up services are already specified in the *docker-compose.yml* file, all we have to do is remote into the production machine through SSH, clone the repo with *git clone https://github.com/winglab/mda-cycling.git*, build the image locally with *docker build . -t mda-cycling* and then start all the services with *docker compose up -d*.

If one would like to check the logs for troubleshooting, one can easily use the command *docker compose logs <service-name>* to view all the logs. Other common commands for MLOps include *docker volume ls* to show all the volumes, *dokcer images* to show all the images etc.

3 Concluding Remarks

This forecasting and monitoring system provides AWV with an automated, actionable approach to bike road management. By balancing maintenance urgency with weather patterns and traffic constraints, this tool ensures vital maintenance can be planned and carried out efficiently, without interrupting civilian mobility.

In the future, this modeling system can be seamlessly expanded to include car roads and even train routes to create a more complete and efficient maintenance scheduling ecosystem.